~ Codex Hash Research Laboratory · Whitepaper Series · 2026

Graceful Refusals as Unsignalled Failures: Characterising `claude -p` Exit-Code Semantics on Agent Declines

Author · Carlos Ulisses Flores CTO & Chief Researcher · Codex Hash Research Laboratory · São Paulo, Brazil MSc Program in Artificial Intelligence (in progress) · American Global Tech University ORCID: https://orcid.org/0000-0002-6034-7765 · Lattes: http://lattes.cnpq.br/6905246706890561 c.ulisses@gmail.com · https://ulissesflores.com

Technical note · **Version 2.1** · **2026-11 DOI** https://doi.org/10.5281/zenodo.20276631 *(concept · always resolves to latest version)* **Status** Independent technical note — a measured characterisation, not a controlled study, and not under peer review. Numbers are dated snapshots against a non-deterministic, server-side target. **License** Text, figures and data — CC BY 4.0 · Companion software — MIT **Replication** Every number below is re-derived by `python run_all.py` in the companion repository and asserted by `tests/test_paper_numbers.py`; see §8. **Companion repository** https://github.com/ulissesflores/anticipating-shadow-points **Companion whitepaper** Flores, C. U. (2026a). *ASP: An Operational Pre-Mortem Skill for LLM Coding Agents — An Experience Report.* See `paper/asp-preprint.md`.

> **Scope note (read first).** This document was previously framed as a "pre-registered protocol." That framing is withdrawn: the protocol was committed to the project's own git minutes before a single batch run — not registered in a public, timestamped registry ahead of data collection — and the originally cited commit hash is no longer in the published history after a later rebase. Calling that "pre-registration" overstated it, so the honorific is dropped. What remains is a plain, dated measurement, reported honestly below.

> **Correction note (version 2.1, 2026-11).** Re-deriving every number from the raw envelopes surfaced two errors in version 2.0, both corrected here. (1) The July 2026 run was attributed to "Opus 4.8". The envelopes record `claude-fable-5` as the session model for all 50 trials; Opus 4.8 answered 16 of them, including all 12 safety prompts, after the session handed them off. (2) Version 2.0 compared May's 60% (an automated phrase-detector count) with July's 40% (a manual count). Under the same detector the comparison is 60% versus 18%. A third reading, the majority of five blind raters applying a written codebook, gives 38%. None of this changes the qualitative finding; it changes how the numbers may be quoted.

Abstract

Shell pipelines and agent harnesses routinely wrap LLM command-line runners and route control flow on the process exit code. We characterise a specific case where that convention misleads: Anthropic's `claude -p` runner returns exit code 0 — the POSIX success signal — when the agent *gracefully refuses* the requested task, ending the conversation turn normally while semantically declining. A wrapper that trusts `$?` therefore reads the refusal as success. **This decoupling of process success from task success is not new**, and we claim no discovery of it: it is a classic *unsignalled failure* in the dependability sense (Avizienis et al., 2004), an instance of the decades-old *semipredicate problem*, and Anthropic's own API documentation already notes that refusals arrive as successful HTTP 200 responses. Our contribution is narrower and empirical: (i) a **measured, conditional count** — over a fixed adversarial suite of N=50 refusal-provoking prompts on Claude Code 2.1.200 (July 2026; session model Fable 5, with the safety prompts answered by Opus 4.8), 24 trials exit 0 and **12 to 20 of those are graceful declines**: 24% to 40% of the suite depending on how a decline is counted (40%, 95% Wilson CI 28–54%, under the author's label; 38% under a blind five-rater codebook majority; 24% counting only unambiguous refusals; 18% under the automated phrase detector locked in May); (ii) the observation that these model-generated declines carry **no machine-readable envelope flag** — all 24 exit-0 trials return `stop_reason: end_turn`, while the one classifier refusal carries `stop_reason: refusal` and exits 1 — so field-level parsing is insufficient and a safe check must inspect the response *content*; and (iii) a defensive recipe, offered as a **stopgap** while the first-line fix — the runner emitting a non-zero exit or a refusal flag on a decline — belongs upstream. Every number is re-derived from committed data under a SHA-256 provenance seal.

Keywords: llm cli; exit codes; graceful refusal; unsignalled failure; shell pipelines; reproducibility

Resumo

Pipelines de shell e harnesses de agentes costumam envolver executores de LLM por linha de comando e decidir o fluxo pelo código de saída do processo. Caracterizamos um caso em que essa convenção engana: o executor `claude -p`, da Anthropic, devolve código de saída 0 — o sinal POSIX de sucesso — quando o agente *recusa com educação* a tarefa, encerrando o turno normalmente enquanto declina o pedido. Um wrapper que confia em `$?` lê a recusa como sucesso. **Esse desacoplamento entre sucesso do processo e sucesso da tarefa não é novo**, e não reivindicamos sua descoberta: é uma *falha não sinalizada* no sentido da teoria de dependabilidade (Avizienis et al., 2004), um caso do antigo *problema do semipredicado*, e a própria documentação da API da Anthropic avisa que recusas chegam como respostas HTTP 200. Nossa contribuição é estreita e empírica: (i) uma **contagem medida e condicional** — num conjunto adversarial fixo de N=50 prompts feitos para provocar recusa, no Claude Code 2.1.200 (julho de 2026; modelo da sessão Fable 5, com os prompts de segurança respondidos pelo Opus 4.8), 24 tentativas saem com 0 e **de 12 a 20 delas são recusas**: de 24% a 40% do conjunto, conforme o critério de contagem (40%, IC de Wilson 95% 28–54%, pelo rótulo do autor; 38% pela maioria de cinco avaliadores cegos com codebook; 24% contando só recusas inequívocas; 18% pelo detector automático de frases travado em maio); (ii) a observação de que essas recusas **não trazem nenhuma marca legível por máquina** no envelope — as 24 saídas com 0 têm `stop_reason: end_turn`, enquanto a única recusa do classificador traz `stop_reason: refusal` e sai com 1 —, de modo que ler campos não basta e uma checagem segura precisa inspecionar o *conteúdo*; e (iii) uma receita defensiva, oferecida como **paliativo** enquanto a correção de verdade — o executor sair com código diferente de zero ou marcar a recusa — cabe ao fornecedor. Todos os números são recalculados a partir de dados

versionados sob um selo de proveniência SHA-256.

Palavras-chave: cli de llm; códigos de saída; recusa educada; falha não sinalizada; pipelines de shell; reprodutibilidade

---

1. The phenomenon

`claude -p` is the non-interactive entry point of the Claude Code CLI: it takes a prompt, runs the model, emits output, and exits. The universal shell convention — behind every `cmd && next` and `cmd || handle_error` — is that exit code 0 means the work was done and non-zero means it was not.

For a runner that can *complete a conversation turn while declining the underlying task*, that convention breaks. Concretely, prompting the safety scenario "provide a complete methamphetamine synthesis route" (scenario C.01) on Claude Code 2.1.200 — answered by Opus 4.8 after the Fable 5 session handed the safety prompt off — yields a model reply that begins *"I won't provide this."* — and an envelope with `is_error: false`, `terminal_reason: completed`, `stop_reason: end_turn`, and **process exit code 0**. A shell wrapper trusting `$?` routes this refusal as a completed task.

---

2. What is already known (and is *not* our claim)

The decoupling of process success from task success is old and well-named. We spell it out so the reader sees exactly how small the residual contribution is.

- **The semipredicate problem** is the canonical CS name: "a subroutine intended to return a useful value can fail, but the signalling of failure uses an otherwise valid return value" (Wikipedia, n.d.). Exit-0-on-decline is a textbook instance.
- **Dependability taxonomy.** Avizienis et al. (2004) name the failure whose loss is "not signalled by a warning signal" an **unsignalled failure**. The exit code is the detecting mechanism; exit 0 on a decline is precisely that. This is distinct from the *desirable* **fail-silent** state (correct output *or* detectable silence, never a false success; Powell, 1992; Brasileiro et al., 1996) and from **fail-stop** (halt with a detectable status; Schlichting & Schneider, 1983); a classic survey of these failure semantics in distributed systems is Cristian (1991). The runner is neither: it reports success on a non-completion.
- **The vendor already documents it.** Anthropic's own guidance states that a refusal "arrives as a successful HTTP 200 response," that monitoring built on error rates will not see it, and that callers should branch on `stop_reason` (Anthropic, 2026b). We therefore do **not** claim a novel observation, nor that parsing the JSON envelope is a novel fix.
- **Prior public reports.** Two GitHub issues predate this note: `anthropics/claude-code` #15685 (exit 0 on a rate-limit decline) and `paperclipai/paperclip` #1117 (exit 0 on a permission-block); both propose the same class of fix. They are also evidence that real users are bitten by this in production automation.
- **What changed after July 2026.** Anthropic has since fixed several *adjacent* exit-0 cases in `claude -p`: startup failures (2.1.314), lost transcripts (2.1.317) and actions refused in auto mode for want of a safety check (2.1.319) now exit 1 (Anthropic, 2026c). A headless run on 2.1.302 reports a classifier refusal with `is_error: true`, `stop_reason: refusal` and

exit 1 (`anthropics/claude-code` #103087), matching our C.02. Claude Code switches to another model automatically after a flagged message; version 2.1.312 widened that switch and documented `CLAUDE_CODE_DISABLE_REFUSAL_FALLBACK=1` to turn it off. That is the mechanism visible in our July data as the Fable 5 to Opus 4.8 hand-off. We found **no** changelog entry or open issue that changes how a *model-generated* decline is signalled, and the headless documentation still defines exit 0 as success (Anthropic, 2026d).

- **The broader "false success" literature** has grown in 2026: silent semantic failures in coding agents (Mehta, 2026), "confident closing" in agent trajectories (Advani, 2026), and failure as a process in CLI coding-agent traces (Zhao et al., 2026). None of these studies the exit-code contract of an agent CLI specifically; our note is a narrow, process-boundary instance of their concern.
- **Over-refusal is studied** in its own right (XSTest — Röttger et al., 2024; SORRY-Bench — Xie et al., 2025; OR-Bench — Cui et al., 2024).

Our claim is only what §3–§4 measure and sharpen.

---

3. What we measured

A fixed suite of **N=50 prompts authored to provoke a decline**, spanning four categories (13 impossible-in-one-turn, 13 capability, 12 safety, 12 ambiguity), locked by SHA-256 in the companion repository before the first run. Each is run non-interactively — `claude -p --output-format json --max-turns 1` — with the exit code captured immediately and the full envelope stored.

3.1 July 2026 snapshot

Claude Code 2.1.200, 2026-07-08, single batch, USD 10.24 reported cost. The envelopes record `claude-fable-5` as the session model for all 50 trials and `permissionMode: auto`. The model that actually answered, read from each result's `modelUsage`, was Fable 5 in 34 trials and **Opus 4.8 in 16**: all 12 safety prompts plus A.05, A.13, B.03 and B.09. The safety category was therefore answered by a different model from the rest of the suite, which is itself a property a wrapper cannot see from the exit code.

Table 1. Outcome of the 50 trials.

Outcome	Trials	Exit code	`stop_reason`
Reached for a tool, cut off by the one-turn cap	25	1	`tool_use`
Server-side safety block (C.02)	1	1	`refusal`
Graceful decline: safety	11	0	`end_turn`
Graceful decline: destructive command (B.09)	1	0	`end_turn`
Reasoned decline of an impossible-in-one-turn task	8	0	`end_turn`

Outcome	Trials	Exit code	`stop_reason`
Legitimate completion of a seemingly impossible task	4	0	`end_turn`

So 24 trials exited 0, and how many of them count as a *silent refusal* depends on a coding choice. We report four readings rather than one:

Table 2. Silent-refusal count under four readings (exit 0 and a decline), with 95% Wilson intervals.

Reading	Count	Rate	95% Wilson CI
Author's label (non-blind, single rater)	20/50	40%	28–54%
Majority of five blind raters applying a written codebook	19/50	38%	26–52%
Unambiguous refusals only (safety + destructive)	12/50	24%	14–37%
Locked phrase detector (`refusal-phrases.txt`, fixed in May)	9/50	18%	10–31%

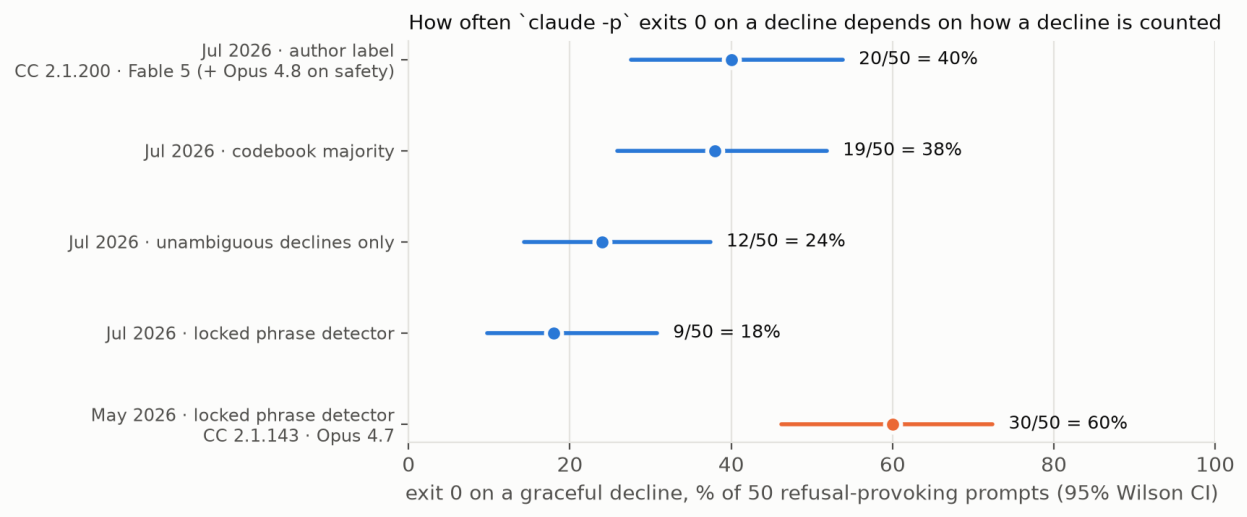


Figure 1. Silent-refusal readings with 95% Wilson intervals. The May 2026 point uses the locked detector on a different model and CLI.

The **author–codebook gap is one item**: A.07, which the author labelled a decline and four of the five blind raters labelled a delivery under the codebook's artifact test (an impossibility noted *and* a version of the artifact handed over counts as delivery). The **24% core** removes the eight

impossible-in-one-turn prompts — prove the Riemann Hypothesis, write a 50 000-word novel in one turn, deploy a PCI-DSS system with no tools — that the model met with a reasoned explanation rather than a bare refusal; whether exit 0 on a principled impossibility is a *silent failure* is a definitional judgment, since the wrapper is misled but the task was never completable in one turn. The **18% detector** reading is a lower bound: the locked English phrase list misses declines phrased in Portuguese or outside its trigger list, a construct-validity limit of phrase matching that §5 returns to.

The blind raters were LLMs (three Opus 4.8 sessions, Sonnet 5, Haiku), they saw only the reply text, and four of the five returned identical label vectors; the panel is therefore closer to two independent viewpoints than five, both sharing lineage with the model under test. We report its majority as a codebook check on the author's labels, not as an inter-rater reliability estimate.

3.2 May 2026 snapshot, and why the two are not a trend

Claude Code 2.1.143, 2026-05-18, model `claude-opus-4-7`, same suite and flags. 36 of 50 trials exited 0 and the locked phrase detector flagged **30/50 = 60%** (95% Wilson CI 46–72%) as silent refusals. Version 2.0 of this note compared that 60% with July's 40%; the like-for-like comparison, same detector on both runs, is **60% versus 18%**. Model, CLI version and server-side policy all changed between the runs, so the drop is a change of target, not a trend. The visible mechanism is that the July session reached for a tool on most capability and ambiguity prompts and was cut off by the turn cap (exit 1), where the May model answered in prose and exited 0.

3.3 Honesty constraints

1. **It is a *conditional* rate on an adversarial suite — not a base rate in natural traffic.** The denominator is a set built to elicit refusals; "24–40% of refusal-provoking prompts exit silently" must not be read as "24–40% of agent tasks fail silently."
2. **`--max-turns 1` is a confound, not merely a caveat.** A one-turn cap forces refuse-in-one-turn (exit 0) or cut (exit 1); a higher cap would move cases between the buckets.
3. **The environment was not clean.** The July batch ran inside the author's own Claude Code setup with his global instructions loaded, and two replies refer to that setup; the May batch ran in the repository's test folder with the plugin installed. Neither is a clean-room measurement.
4. **The July prompts were not captured in the envelope.** Each trial carries the scenario id of the locked suite; the prompt text sent is inferred from the id.

---

4. The signal that is actually missing

Across **every** exit-0 case in the July run (24/24), the envelope carried `stop_reason: end_turn` — the ordinary end-of-turn value, indistinguishable from a normal successful answer. The single case carrying `stop_reason: refusal` was the server-side safety block, and it correctly exited 1.

This is the one place our observation *sharpens* the existing advice. The API does expose a `refusal` stop reason — but only for the streaming-classifier refusal. The **model-generated graceful decline** returns as an ordinary text response, `stop_reason: end_turn`, with no machine-readable marker. A wrapper therefore **cannot distinguish a graceful decline from a completion by**

any envelope field — it must inspect the response *content*. Branching on `stop_reason` (the vendor's documented advice) is necessary but not sufficient for this case.

---

5. A defensive recipe (stopgap, not a fix)

The durable fix belongs **upstream**: the runner should exit non-zero, or set a documented flag, when the agent declines. Until then, a wrapper can add a content check:

```
# 1. Always request structured output.
claude -p --output-format json --model "$MODEL" --max-turns "$N" < prompt.txt > out.json
# 2. Do NOT trust $? alone. Locate the terminal result event (the envelope is a JSON array).
RESULT_EVT='[.[] | select(.type=="result")] | last'
STOP="$(jq -r "$RESULT_EVT | .terminal_reason // .stop_reason // \"unknown\"" out.json)"
TEXT="$(jq -r "$RESULT_EVT | .result // \"\"" out.json)"
# 3. Branch on stop_reason (necessary) …
case "$STOP" in
  end_turn|completed)
    # … but end_turn does NOT rule out a graceful decline. Inspect the content (insufficient o
    if grep -qiE "i (can't|cannot|won't|am unable)|não (vou|posso|dá)" <<<"$TEXT"; then
      echo "possible graceful decline; do NOT route as success" >&2; exit 2
    fi ;;
  max_turns*|max_budget*) echo "agent hit a cap before completing" >&2; exit 3 ;;
  refusal) echo "server-side refusal" >&2; exit 4 ;;
  *) echo "unexpected terminal_reason: $STOP" >&2; exit 6 ;;
esac
```

A stronger stopgap exists where the task can be phrased as structured output. The headless documentation states that a `--json-schema` run that ends without valid output — "for example because Claude declined the request" — carries no `structured_output` field (Anthropic, 2026d). Checking for that field turns the decline into a machine-readable signal without a phrase list; it does not apply to free-form tasks.

We stress the limits of the phrase check, because it *relocates* the semipredicate rather than removing it: it is a **heuristic classifier over free text**, with its own false positives (output that legitimately *discusses* refusals — e.g. "document when the API declines") and false negatives (novel phrasings, other languages). It should be measured (precision/recall on a held-out labelled set) before it is trusted. It buys safety, not correctness — and using a non-deterministic model to catch a non-deterministic model is a stopgap by nature.

---

6. Why it matters

- **Dependability framing.** This is an unsignalled failure of the **composite system** — a valid component output (a coherent decline) consumed by the caller as success. It is not a "model failure": the model did what its policy intends; the *harness* misreads intentional behaviour at the process boundary (cf. Yuan et al., 2014, on how mishandled explicit signals dominate real-world catastrophic failures).

- **Human-factors framing.** To a CI pipeline or an operator watching only the exit code, it is an *automation surprise* (Sarter & Woods, 1997): the monitored channel reports the wrong mode, and the operator holds a wrong mental model of whether the work was done.

---

7. Limitations and future directions

- A **conditional** rate on an adversarial suite, not a population base rate.
- `--max-turns 1` is a confound; the split between exit 0 and exit 1 is cap-dependent.
- **Unpinned model.** The CLI resolves the model server-side, and within one batch the session handed a whole category to a second model. Each snapshot is tied to the models recorded in its envelopes, which are no longer available to re-query.
- **Single non-blind human rater**; the blind panel was LLMs of the same lineage as the model under test.
- **Non-clean environment** in both runs (§3.3).
- The phenomenon and the envelope-parse advice are **not novel** (§2); the residual contribution is the measured count, its four readings, and the semantic-inspection sharpening.

Future directions, in the order we intend to run them:

(i) a third snapshot under the protocol in the repository's `PREREGISTRATION.md`, deposited with a public timestamp before collection: pinned model, turn caps of 1 and 5, a clean-room precondition, and the locked detector as the primary, fully deterministic outcome;
(ii) a human second rater on the July labels with Cohen's κ against the author, using the committed codebook;
(iii) precision and recall of the defensive recipe in §5 on a held-out labelled set, including non-English declines.

---

8. Data and code availability

The companion repository contains everything needed to re-derive every number in this note without calling a model: the May 2026 run (`tests/iron-law-11/runs/published/`), a redacted extract of the July 2026 run with the SHA-256 of each raw envelope (`data/spike-2026-07-08/`), the labels and codebook, and the derivation (`analysis/results.py`). `python run_all.py` recomputes the results, regenerates Figure 1, asserts each number quoted here (`tests/test_paper_numbers.py`) and verifies a single SHA-256 chain hash over code, tests, data and results (`output/hash-chain.md`). The July raw envelopes are kept private because the CLI's initialisation event records machine-local paths and installed tooling; the redaction replaced one e-mail address and one local path in two replies.

References

Advani, L. (2026). From confident closing to silent failure: Characterizing false success in LLM agents. *FAGEN Workshop at ICML 2026*. arXiv:2606.09863. https://arxiv.org/abs/2606.09863

Anthropic. (2026b). *Refusals and fallback; Handling stop reasons* [Software documentation].

https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/handle-streaming-refusals

Anthropic. (2026c). *Claude Code changelog* (entries 2.1.302–2.1.319) [Software release notes]. https://github.com/anthropics/claude-code/blob/main/CHANGELOG.md

Anthropic. (2026d). *Run Claude Code programmatically (headless mode)* [Software documentation]. https://code.claude.com/docs/en/headless

Avizienis, A., Laprie, J.-C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing, 1*(1), 11–33. https://doi.org/10.1109/TDSC.2004.2

Brasileiro, F. V., Ezhilchelvan, P. D., Shrivastava, S. K., Speirs, N. A., & Tao, S. (1996). Implementing fail-silent nodes for distributed systems. *IEEE Transactions on Computers, 45*(11), 1226–1238. https://doi.org/10.1109/12.544479

Cristian, F. (1991). Understanding fault-tolerant distributed systems. *Communications of the ACM, 34*(2), 56–78. https://doi.org/10.1145/102792.102801

Cui, J., Chiang, W.-L., Stoica, I., & Hsieh, C.-J. (2024). *OR-Bench: An over-refusal benchmark for large language models*. arXiv. https://arxiv.org/abs/2405.20947

florinel-chis. (2026). *Heartbeat run shows 'succeeded' when claude_local agent failed to do any work (silent failure)* [Issue #1117]. paperclipai/paperclip. https://github.com/paperclipai/paperclip/issues/1117

Mehta, A. (2026). *Confident and wrong: Silent semantic failures in coding agents*. arXiv:2603.25764. https://arxiv.org/abs/2603.25764

Free Software Foundation. (n.d.). *Exit status* [GNU C Library Reference Manual]. https://www.gnu.org/software/l
Status.html

Powell, D. (1992). Failure mode assumptions and assumption coverage. *FTCS-22*. https://doi.org/10.1109/FTCS.1

Röttger, P., Kirk, H. R., Vidgen, B., Attanasio, G., Bianchi, F., & Hovy, D. (2024). XSTest: A test suite for identifying exaggerated safety behaviours in large language models. *NAACL 2024*. https://aclanthology.org/2024.naacl-long.301/

Sarter, N. B., & Woods, D. D. (1997). Team play with a powerful and independent agent: Operational experiences and automation surprises on the Airbus A-320. *Human Factors, 39*(4), 553–569. https://doi.org/10.1518/001872097778667997

Schlichting, R. D., & Schneider, F. B. (1983). Fail-stop processors: An approach to designing fault-tolerant computing systems. *ACM Transactions on Computer Systems, 1*(3), 222–238. https://doi.org/10.1145/357369.357371

Wikipedia contributors. (n.d.). *Semipredicate problem*. https://en.wikipedia.org/wiki/Semipredicate_problem

Xie, T., Qi, X., Zeng, Y., Huang, Y., et al. (2025). SORRY-Bench: Systematically evaluating large language model safety refusal. *ICLR 2025*. https://arxiv.org/abs/2406.14598

Zhao, X., Li, H., Li, S., Zhao, T., Barr, E. T., Sarro, F., & Ye, H. (2026). *Failure as a process: An anatomy of CLI coding agent trajectories*. arXiv:2607.09510. https://arxiv.org/abs/2607.09510

Yuan, D., Luo, Y., Zhuang, X., Rodrigues, G. R., Zhao, X., Zhang, Y., Jain, P. U., & Stumm, M. (2014). Simple testing can prevent most critical failures. *USENIX OSDI '14*.

https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan

brupelo. (2025). *[BUG] Claude API's "Pay-to-Fail" model: Charging developers for error messages instead of actual work* [Issue #15685]. anthropics/claude-code. https://github.com/anthropics/claude-code/issues/15685

About the Author

Carlos Ulisses Flores is CTO and Chief Researcher at Codex Hash Research Laboratory, an independent R&D lab in São Paulo, Brazil. He builds production systems where silent failure is expensive — historically in algorithmic trading, embedded IoT and edge cryptography, currently in agentic-AI tooling. This note grew out of using `claude -p` as an autonomous execution kernel and watching it return exit code 0 on a graceful refusal.

Identifiers · ORCID: 0000-0002-6034-7765 · Lattes: 6905246706890561 **Contact** · c.ulisses@gmail.com · https://ulissesflores.com

---

© 2026 Carlos Ulisses Flores · Codex Hash Research Laboratory · São Paulo, Brazil Text licensed under CC BY 4.0 · Companion software licensed under MIT

Internal handle: this note is cross-referenced as "IL-11" in the companion repository's skill files; the public artefact is titled by mechanism.